

第 5 章：索引

2026 版

邹兆年

哈尔滨工业大学
计算学部
海量数据计算研究中心
电子邮件: znzou@hit.edu.cn

2026 春

教学内容¹

① 5.1 索引的概念

② 5.2 有序索引

- 5.2.1 B+ 树的结构
- 5.2.2 B+ 树查找
- 5.2.3 B+ 树插入
- 5.2.4 B+ 树删除
- 5.2.5 B+ 树的并发访问
- 5.2.6 B+ 树的优化

③ 5.3 哈希索引

- 5.3.1 外存哈希表
- 5.3.2 可扩展哈希表
- 5.3.3 线性哈希表

¹课件更新于 2026 年 5 月 11 日

5.1 索引的概念

索引 (Index)

索引能够帮助 DBMS 快速找到关系中满足搜索条件的元组 [演示](#)

例 (索引)

索引		Student 关系					
Sname	元组记录编号 (RID)	RID	Sno	Sname	Ssex	Sage	Sdept
Abby	rid_3	rid_1	CS-001	Elsa	F	19	CS
Ed	rid_2	rid_2	CS-002	Ed	M	19	CS
Elsa	rid_1	rid_3	MA-001	Abby	F	18	Math
Nick	rid_4	rid_4	PH-001	Nick	M	20	Physics

```
SELECT Sdept FROM Student WHERE Sname = 'Elsa';
```

- 如果没有索引，则只能通过扫描 Student 关系来完成查询
- 如果有上述索引，则可以通过该索引来快速找到元组

索引的基本概念

- 索引键 (Index Key): 索引检索元组时使用的属性列表
- 索引项 (Index Entry): (元组索引键值, 元组 RID)
- 索引 (Index): 将元组的索引键值映射到元组 RID 的数据结构

例 (索引)

索引		RID	Student 关系				
Sname	RID		Sno	Sname	Ssex	Sage	Sdept
Abby	rid ₃	rid ₁	CS-001	Elsa	F	19	CS
Ed	rid ₂	rid ₂	CS-002	Ed	M	19	CS
Elsa	rid ₁	rid ₃	MA-001	Abby	F	18	Math
Nick	rid ₄	rid ₄	PH-001	Nick	M	20	Physics

索引的分类

按索引实现方式分为:

- 有序索引 (Ordered Index): 按索引键有序排列索引项
- 哈希索引 (Hash Index): 按索引键的哈希值分桶组织索引项

有序索引 (Ordered Index)

按照索引键有序排列索引项

例 (有序索引)

有序索引		RID	Student 关系				
Sname ↑	RID		Sno	Sname	Ssex	Sage	Sdept
CS-001	rid ₁	rid ₁	CS-001	Elsa	F	19	CS
CS-002	rid ₂	rid ₂	CS-002	Ed	M	19	CS
MA-001	rid ₃	rid ₃	MA-001	Abby	F	18	Math
PH-001	rid ₄	rid ₄	PH-001	Nick	M	20	Physics

哈希索引 (Hash Index)

按索引键的哈希值分桶组织索引项

- 键为 K 的索引项属于编号为 $hash(K)$ 的桶, 其中 $hash$ 为哈希函数
- 哈希索引仅支持索引键上的等值查找

例 (哈希索引)

哈希索引		RID	Student 关系				
桶 0	hash(Sdept)		Sno	Sname	Ssex	Sage	Sdept
hash('Math') = 0	rid ₃	rid ₃	MA-001	Abby	F	18	Math
桶 1	hash(Sdept)	RID					
hash('CS') = 1	rid ₁	rid ₁	CS-001	Elsa	F	19	CS
hash('CS') = 1	rid ₂	rid ₂	CS-002	Ed	M	19	CS
桶 2	hash(Sdept)	RID					
hash('Physics') = 2	rid ₄	rid ₄	PH-001	Nick	M	20	Physics

有序索引的分类（一）

按照索引项 RID 顺序与元组 RID 顺序是否一致，可分为：

- 聚簇索引 (Clustered Index): 索引项 RID 顺序 = 元组 RID 顺序
- 非聚簇索引 (Nonclustered Index): 索引项 RID 顺序 $\neq$ 元组 RID 顺序

聚簇索引 (Clustered Index)

索引项 RID 顺序 = 元组 RID 顺序

- 聚簇索引的键通常是关系的主键
- 一个关系上通常只有一个聚簇索引（为什么？） [演示](#)

例 (聚簇索引)

聚簇索引		RID	Student 关系				
Sno	RID		Sno	Sname	Ssex	Sage	Sdept
CS-001	<i>rid₁</i>	<i>rid₁</i>	CS-001	Elsa	F	19	CS
CS-002	<i>rid₂</i>	<i>rid₂</i>	CS-002	Ed	M	19	CS
MA-001	<i>rid₃</i>	<i>rid₃</i>	MA-001	Abby	F	18	Math
PH-001	<i>rid₄</i>	<i>rid₄</i>	PH-001	Nick	M	20	Physics

非聚簇索引 (Nonclustered Index)

索引项 RID 顺序 $\neq$ 元组 RID 顺序

- 一个关系上可以有多个非聚簇索引

例 (非聚簇索引)

非聚簇索引		RID	Student 关系				
Sname	RID		Sno	Sname	Ssex	Sage	Sdept
Abby	<i>rid₃</i>	<i>rid₁</i>	CS-001	Elsa	F	19	CS
Ed	<i>rid₂</i>	<i>rid₂</i>	CS-002	Ed	M	19	CS
Elsa	<i>rid₁</i>	<i>rid₃</i>	MA-001	Abby	F	18	Math
Nick	<i>rid₄</i>	<i>rid₄</i>	PH-001	Nick	M	20	Physics

有序索引的分类（二）

按照关系中每个元组在索引中是否有对应的索引项，可分为：

- 稠密索引 (Dense Index): 每个元组都有对应的索引项
- 稀疏索引 (Sparse Index): 不是每个元组都有对应的索引项

稠密索引 (Dense Index)

每个元组都有对应的索引项

- 非聚簇索引一定是稠密索引

例 (稠密索引)

稠密索引		Student 关系					
Sname	RID	RID	Sno	Sname	Ssex	Sage	Sdept
Abby	<i>rid₃</i>	<i>rid₁</i>	CS-001	Elsa	F	19	CS
Ed	<i>rid₂</i>	<i>rid₂</i>	CS-002	Ed	M	19	CS
Elsa	<i>rid₁</i>	<i>rid₃</i>	MA-001	Abby	F	18	Math
Nick	<i>rid₄</i>	<i>rid₄</i>	PH-001	Nick	M	20	Physics

稀疏索引 (Sparse Index)

不是每个元组都有对应的索引项

- 实现方法 1: 为数据文件每页中第一个元组建立索引项
- 实现方法 2: 为数据文件中每个不同索引键值的第一个元组建立索引项
- 聚簇索引通常是稀疏索引

例 (稀疏索引)

稀疏索引		Student 关系				
Sno	RID	Sno	Sname	Ssex	Sage	Sdept
CS-001	<i>rid₁</i>	页 1				
MA-001	<i>rid₃</i>	<i>rid₁</i>				
		<i>rid₂</i>				
		页 2				
		<i>rid₃</i>				
		<i>rid₄</i>				

有序索引的分类 (三)

按照索引键是否为关系的主键, 可分为:

- 主索引 (Primary Index): 索引键是主键
- 二级索引 (Secondary Index): 索引键不是主键

主索引 (Primary Index)

索引键是主键

- 一个关系至多有一个主索引
- 主索引一般是聚簇索引

例 (主索引)

主索引		Student 关系					
Sno	RID	RID	Sno	Sname	Ssex	Sage	Sdept
CS-001	rid ₁	rid ₁	CS-001	Elsa	F	19	CS
CS-002	rid ₂	rid ₂	CS-002	Ed	M	19	CS
MA-001	rid ₃	rid ₃	MA-001	Abby	F	18	Math
PH-001	rid ₄	rid ₄	PH-001	Nick	M	20	Physics

二级索引 (Secondary Index)

索引键不是主键

- 一个关系可以有多个二级索引
- 二级索引通常是非聚簇索引

例 (二级索引)

二级索引		Student 关系					
Sname	RID	RID	Sno	Sname	Ssex	Sage	Sdept
Abby	<i>rid₃</i>	<i>rid₁</i>	CS-001	Elsa	F	19	CS
Ed	<i>rid₂</i>	<i>rid₂</i>	CS-002	Ed	M	19	CS
Elsa	<i>rid₁</i>	<i>rid₃</i>	MA-001	Abby	F	18	Math
Nick	<i>rid₄</i>	<i>rid₄</i>	PH-001	Nick	M	20	Physics

有序索引的分类 (四)

按照索引键的值是否唯一, 可分为:

- 唯一索引 (Unique Index): 索引键值唯一
- 非唯一索引 (Non-unique Index): 索引键值不唯一

唯一索引 (Unique Index)

索引键值唯一

- 主索引一定是唯一索引
- 二级索引不一定是唯一索引

例 (唯一索引)

唯一索引		Student 关系					
Sname	RID	RID	Sno	Sname	Ssex	Sage	Sdept
CS-001	<i>rid₁</i>	<i>rid₁</i>	CS-001	Elsa	F	19	CS
CS-002	<i>rid₂</i>	<i>rid₂</i>	CS-002	Ed	M	19	CS
MA-001	<i>rid₃</i>	<i>rid₃</i>	MA-001	Abby	F	18	Math
PH-001	<i>rid₄</i>	<i>rid₄</i>	PH-001	Nick	M	20	Physics

SQL 索引定义语句

- 创建索引: `CREATE INDEX`
- 创建唯一索引: `CREATE UNIQUE INDEX`
- 删除索引: `DROP INDEX`

索引结构 (Index Structure)

有序索引的数据结构

- B+ 树：广泛用于各种 DBMS (5.2 节)
- 跳表 (Skiplist)：多用于内存数据库系统
- 字典树 (Trie)：多用于内存数据库系统
- 日志结构合并树 (Log-Structured Merge-Tree, LSM-Tree)：广泛用于 NoSQL DBMS 的存储引擎

哈希索引的数据结构

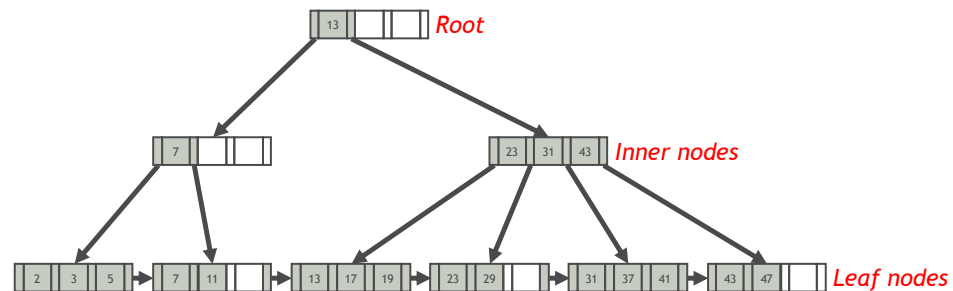
- 外存哈希表 (5.3 节)

5.2 有序索引

B+ 树

M 路完美平衡搜索树 ($M \gg 2$)

- 每个节点恰好存入 1 个页
- 所有叶节点在同一层上
- 除根节点外，每个节点至少“半满”，即 $M/2 - 1 \leq \#keys \leq M - 1^2$

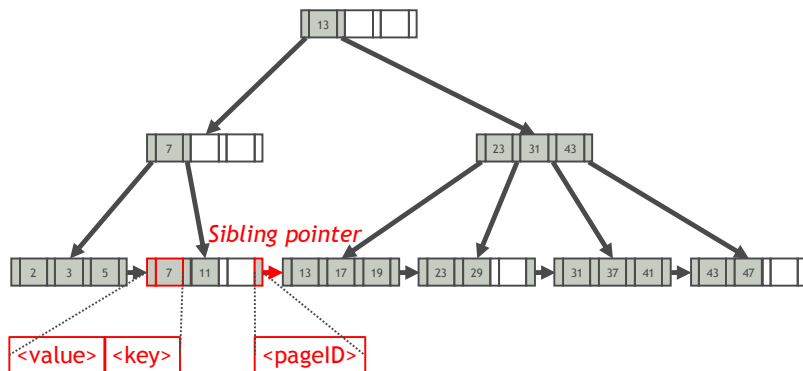


²Raghu Ramakrishnan, Johannes Gehrke. Database Management Systems, 3rd Edition. 2003.

5.2.1 B+ 树的结构

B+ 树叶节点

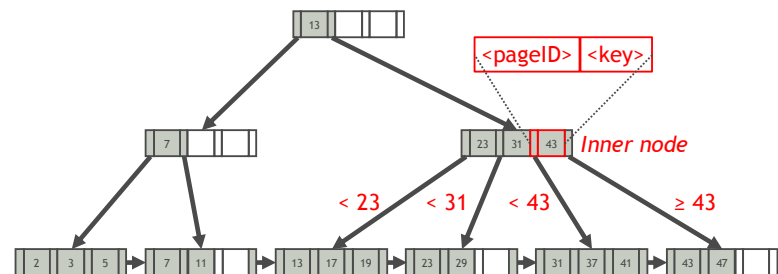
- 索引项数组（索引项按键值升序排序）
- 右侧叶节点的页号（“指针”）



B+ 树内节点

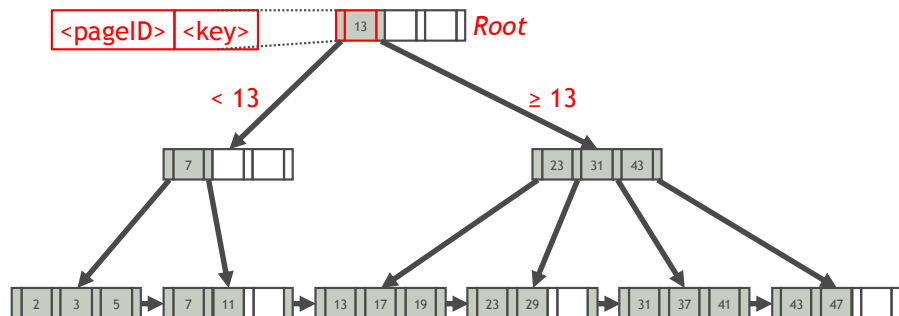
每个内节点包含一个键数组 Key 和一个儿子节点页号（“指针”）数组 Ptr

- Key 中有 k 个非空键值当且仅当 Ptr 中有 $k+1$ 个非空指针
- Key 中键值升序排列
- $Ptr[0]$ 指向的子树中的键值 $< Key[0]$
- $Ptr[k+1]$ 指向的子树中的键值 $\geq Key[k]$
- $Key[i-1] \leq Ptr[i]$ 指向的子树中的键值 $< Key[i]$



B+ 树根节点

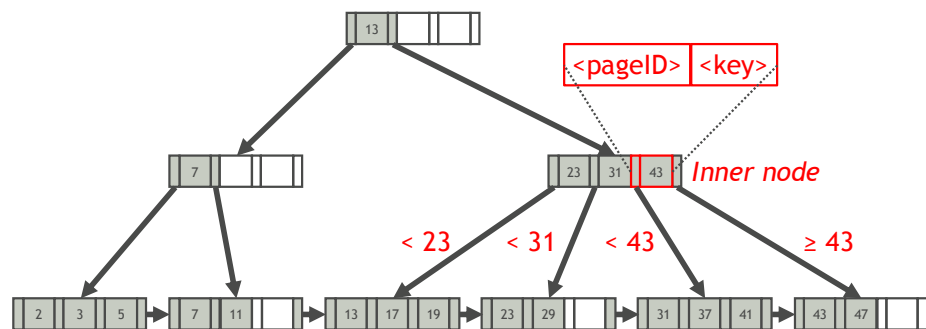
- 结构与内节点相同
- 不必“半满”（包含至少 1 个键值即可）



B+ 树非叶节点键值的性质

定理 (B+ 树非叶节点键值的性质)

对于 $i \geq 1$, $Key[i-1] = Ptr[i]$ 指向的子树的最小键值



5.2.2 B+ 树查找

B+ 树查找

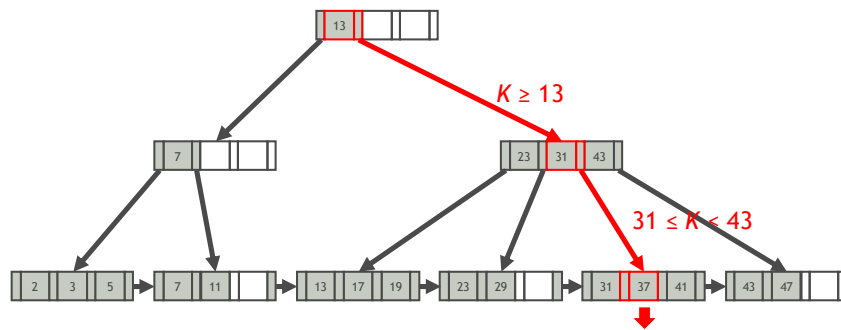
- 点查找：查找键值为 K 的索引项
- 区间查找：查找键值在区间 $[L, U]$ 中的索引项

B+ 树点查找

查找键值为 K 的索引项

- 在根节点和内节点的引导下，找到 K 所在的叶节点
- 在该叶节点中查找键值为 K 的索引项

例：查找 $K = 37$ 的索引项



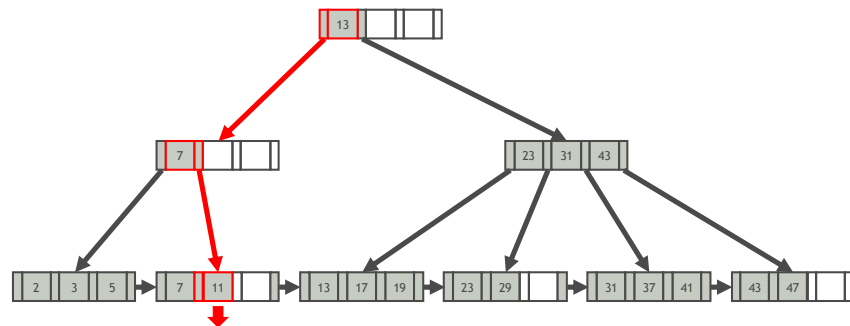
5.2.3 B+ 树插入

B+ 树区间查找

查找键值 $K \in [L, U]$ 的索引项

- 1 找到具有大于等于 L 的最小键值的索引项 E
- 2 遍历 E 右侧的索引项，直至索引项键值大于 U 为止

例: 查找 $K \in [10, 25]$ 的索引项



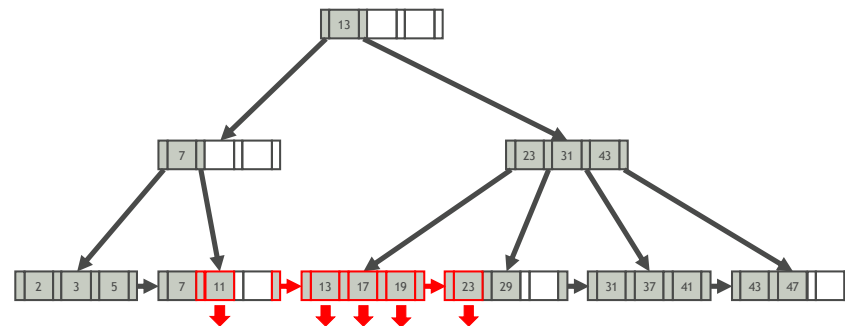
步骤 1/2

B+ 树区间查找

查找键值 $K \in [L, U]$ 的索引项

- 1 找到具有大于等于 L 的最小键值的索引项 E
- 2 遍历 E 右侧的索引项，直至索引项键值大于 U 为止

例: 查找 $K \in [10, 25]$ 的索引项



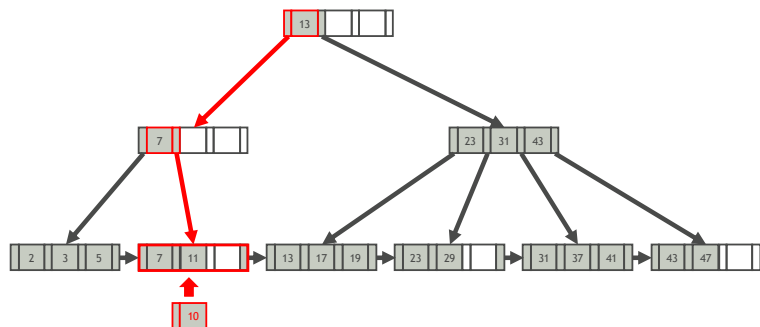
步骤 2/2

插入索引项

插入索引项 (K, rid)

- 1 找到 K 应在的叶节点 L
- 2 将索引项插入 L
- 3 如果 L 不溢出，则插入完成；否则，分裂 L

例 1: 插入 $K = 10$ 的索引项 (无节点溢出)

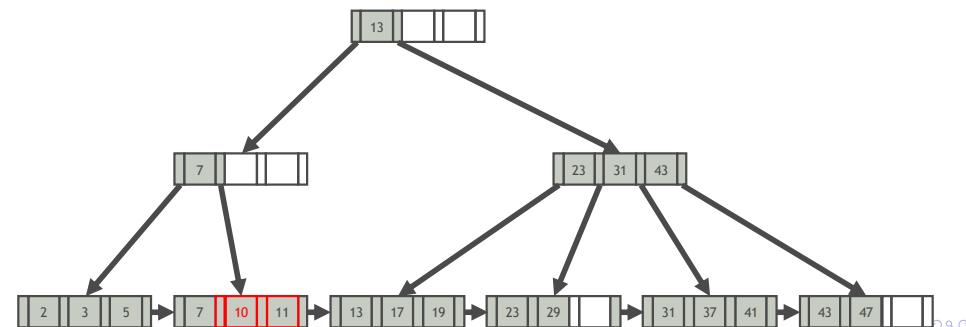


插入索引项

插入索引项 (K, rid)

- 1 找到 K 应在的叶节点 L
- 2 将索引项插入 L
- 3 如果 L 不溢出，则插入完成；否则，分裂 L

例 1: 插入 $K = 10$ 的索引项 (无节点溢出)

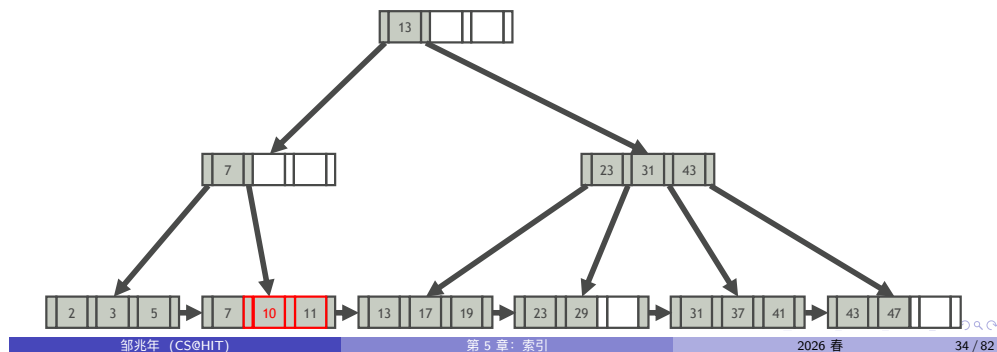


插入索引项

插入索引项 (K, rid)

- 1 找到 K 应在的叶节点 L
- 2 将索引项插入 L
- 3 如果 L 不溢出，则插入完成；否则，分裂 L

例 1: 插入 $K = 10$ 的索引项 (无节点溢出)

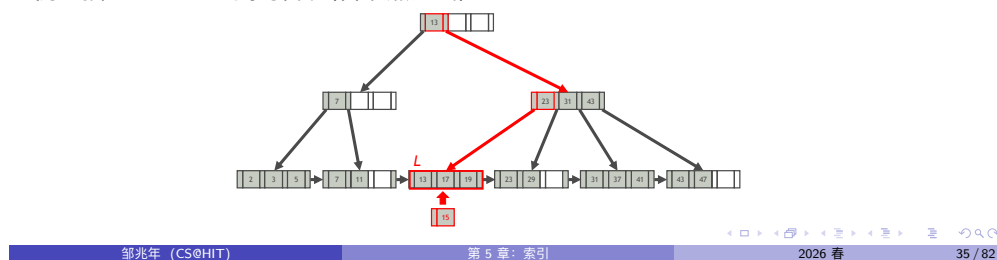


叶节点分裂

插入索引项后导致叶节点 L 分裂

- 1 创建一个新的叶节点 L_2
- 2 将 L 中的索引项平分，前半留在 L 中，后半移入 L_2 中，并将 L_2 中最小的键值存入“中间键 (Middle Key)”变量
- 3 在叶节点链表中，将 L_2 插到 L 的右边
- 4 在 L 的父节点 N 中插入 Middle Key 及指向 L_2 的指针 (L_2 的页号)
- 5 如果 N 不溢出，则完成对 L 的分裂；否则，继续分裂 N

例: 插入 $K = 15$ 的索引项 (含节点溢出)

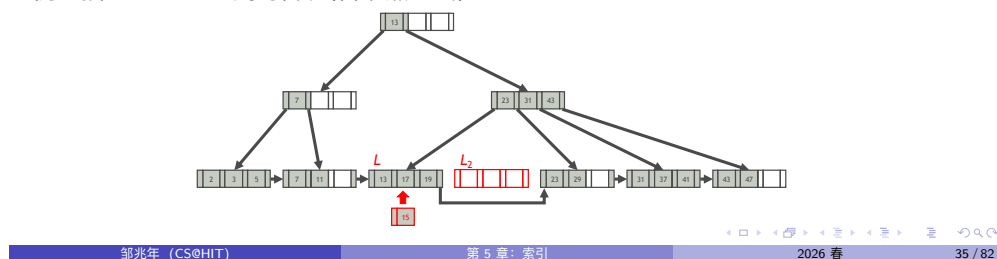


叶节点分裂

插入索引项后导致叶节点 L 分裂

- 1 创建一个新的叶节点 L_2
- 2 将 L 中的索引项平分，前半留在 L 中，后半移入 L_2 中，并将 L_2 中最小的键值存入“中间键 (Middle Key)”变量
- 3 在叶节点链表中，将 L_2 插到 L 的右边
- 4 在 L 的父节点 N 中插入 Middle Key 及指向 L_2 的指针 (L_2 的页号)
- 5 如果 N 不溢出，则完成对 L 的分裂；否则，继续分裂 N

例: 插入 $K = 15$ 的索引项 (含节点溢出)

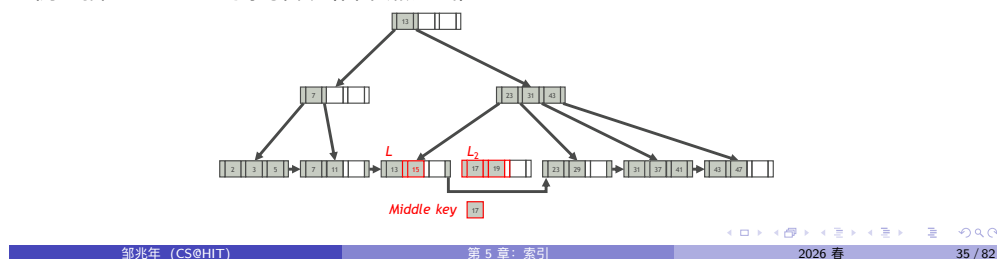


叶节点分裂

插入索引项后导致叶节点 L 分裂

- 1 创建一个新的叶节点 L_2
- 2 将 L 中的索引项平分，前半留在 L 中，后半移入 L_2 中，并将 L_2 中最小的键值存入“中间键 (Middle Key)”变量
- 3 在叶节点链表中，将 L_2 插到 L 的右边
- 4 在 L 的父节点 N 中插入 Middle Key 及指向 L_2 的指针 (L_2 的页号)
- 5 如果 N 不溢出，则完成对 L 的分裂；否则，继续分裂 N

例: 插入 $K = 15$ 的索引项 (含节点溢出)

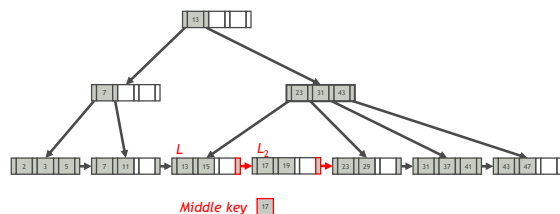


叶节点分裂

插入索引项后导致叶节点 L 分裂

- 1 创建一个新的叶节点 L_2
- 2 将 L 中的索引项平分，前一半留在 L 中，后一半移入 L_2 中，并将 L_2 中最小的键值存入“中间键 (Middle Key)”变量
- 3 在叶节点链表中，将 L_2 插到 L 的右边
- 4 在 L 的父节点 N 中插入 Middle Key 及指向 L_2 的指针 (L_2 的页号)
- 5 如果 N 不溢出，则完成对 L 的分裂；否则，继续分裂 N

例：插入 $K = 15$ 的索引项 (含节点溢出)

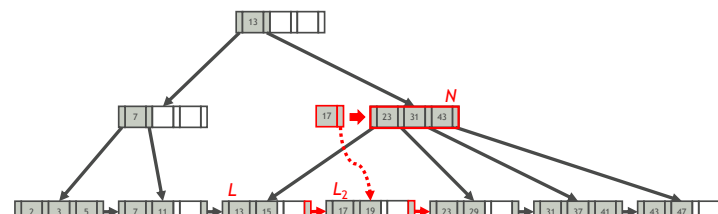


叶节点分裂

插入索引项后导致叶节点 L 分裂

- 1 创建一个新的叶节点 L_2
- 2 将 L 中的索引项平分，前一半留在 L 中，后一半移入 L_2 中，并将 L_2 中最小的键值存入“中间键 (Middle Key)”变量
- 3 在叶节点链表中，将 L_2 插到 L 的右边
- 4 在 L 的父节点 N 中插入 Middle Key 及指向 L_2 的指针 (L_2 的页号)
- 5 如果 N 不溢出，则完成对 L 的分裂；否则，继续分裂 N

例：插入 $K = 15$ 的索引项 (含节点溢出)

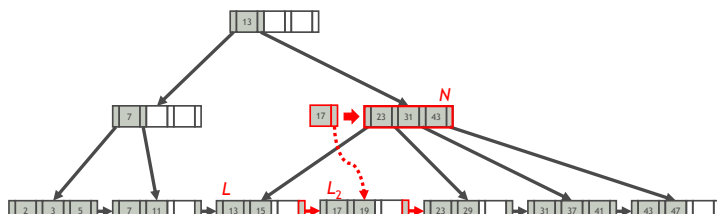


叶节点分裂

插入索引项后导致叶节点 L 分裂

- 1 创建一个新的叶节点 L_2
- 2 将 L 中的索引项平分，前一半留在 L 中，后一半移入 L_2 中，并将 L_2 中最小的键值存入“中间键 (Middle Key)”变量
- 3 在叶节点链表中，将 L_2 插到 L 的右边
- 4 在 L 的父节点 N 中插入 Middle Key 及指向 L_2 的指针 (L_2 的页号)
- 5 如果 N 不溢出，则完成对 L 的分裂；否则，继续分裂 N

例：插入 $K = 15$ 的索引项 (含节点溢出)

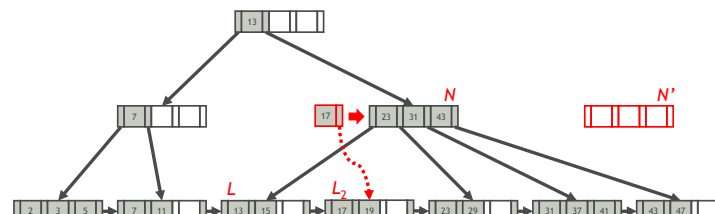


内节点分裂

分裂内节点 N

- 1 创建一个新的内节点 N_2
- 2 将 N 中的“指针”平分，前一半留在 N 中，后一半移入 N_2 中，并将 N 中多余的键值存入“中间键 (Middle Key)”变量
- 3 如果 N 是根节点，则创建一个新的根节点 N' ，并在 N' 中插入一个指向 N 的“指针”
- 4 在 N 的父节点 N' 中插入 Middle Key 及指向 N_2 的“指针” (N_2 的页号)
- 5 如果 N' 不溢出，则完成对 N 的分裂；否则，继续分裂 N'

例：插入 $K = 15$ 的索引项 (含节点溢出)

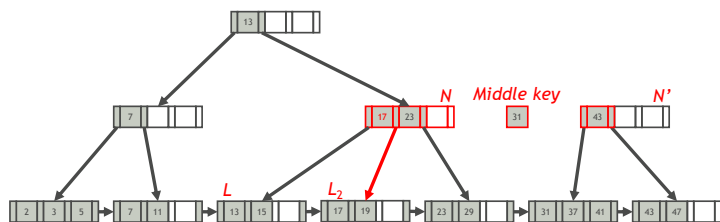


内节点分裂

分裂内节点 N

- 1 创建一个新的内节点 N_2
- 2 将 N 中的“指针”平分，前半留在 N 中，后半移入 N_2 中，并将 N 中多余的键值存入“中间键 (Middle Key)”变量
- 3 如果 N 是根节点，则创建一个新的根节点 N' ，并在 N' 中插入一个指向 N 的“指针”
- 4 在 N 的父节点 N' 中插入 Middle Key 及指向 N_2 的“指针” (N_2 的页号)
- 5 如果 N' 不溢出，则完成对 N 的分裂；否则，继续分裂 N'

例：插入 $K = 15$ 的索引项 (含节点溢出)

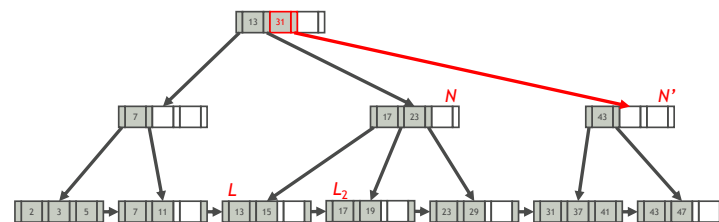


内节点分裂

分裂内节点 N

- 1 创建一个新的内节点 N_2
- 2 将 N 中的“指针”平分，前半留在 N 中，后半移入 N_2 中，并将 N 中多余的键值存入“中间键 (Middle Key)”变量
- 3 如果 N 是根节点，则创建一个新的根节点 N' ，并在 N' 中插入一个指向 N 的“指针”
- 4 在 N 的父节点 N' 中插入 Middle Key 及指向 N_2 的“指针” (N_2 的页号)
- 5 如果 N' 不溢出，则完成对 N 的分裂；否则，继续分裂 N'

例：插入 $K = 15$ 的索引项 (含节点溢出)

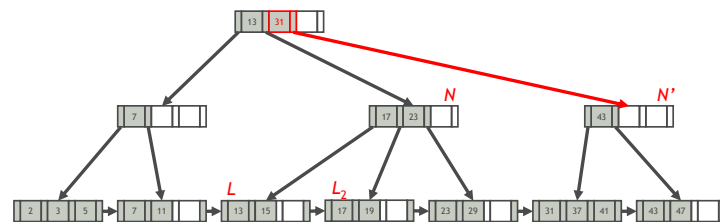


内节点分裂

分裂内节点 N

- 1 创建一个新的内节点 N_2
- 2 将 N 中的“指针”平分，前半留在 N 中，后半移入 N_2 中，并将 N 中多余的键值存入“中间键 (Middle Key)”变量
- 3 如果 N 是根节点，则创建一个新的根节点 N' ，并在 N' 中插入一个指向 N 的“指针”
- 4 在 N 的父节点 N' 中插入 Middle Key 及指向 N_2 的“指针” (N_2 的页号)
- 5 如果 N' 不溢出，则完成对 N 的分裂；否则，继续分裂 N'

例：插入 $K = 15$ 的索引项 (含节点溢出)

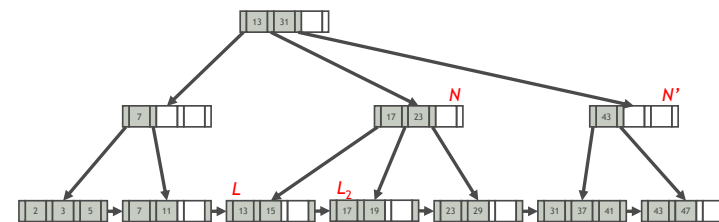


内节点分裂

分裂内节点 N

- 1 创建一个新的内节点 N_2
- 2 将 N 中的“指针”平分，前半留在 N 中，后半移入 N_2 中，并将 N 中多余的键值存入“中间键 (Middle Key)”变量
- 3 如果 N 是根节点，则创建一个新的根节点 N' ，并在 N' 中插入一个指向 N 的“指针”
- 4 在 N 的父节点 N' 中插入 Middle Key 及指向 N_2 的“指针” (N_2 的页号)
- 5 如果 N' 不溢出，则完成对 N 的分裂；否则，继续分裂 N'

例：插入 $K = 15$ 的索引项 (含节点溢出)

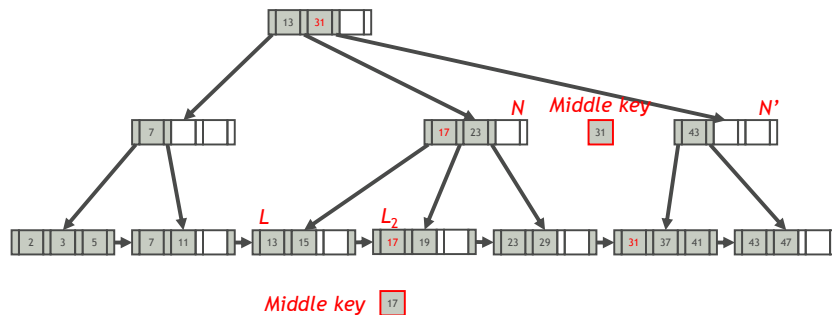


中间键 (Middle Key) 的作用

保持 B+ 树非叶节点键值的性质

定理 (B+ 树非叶节点键值的性质)

对于 $i \geq 1$, $Key[i-1] = Ptr[i]$ 指向的子树的最小键值



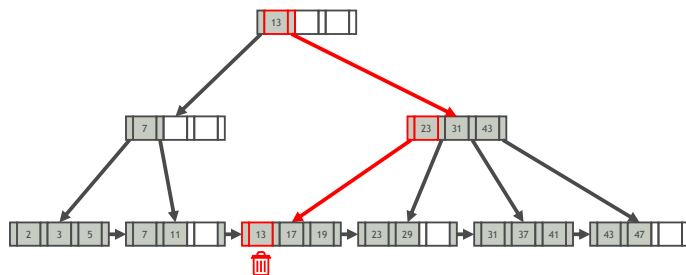
5.2.4 B+ 树删除

删除索引项

删除键为 K 的索引项

- 1 找到 K 所在的叶节点 L
- 2 从 L 中删除键值为 K 索引项
- 3 如果 L 发生下溢 (Underflow), 则需要处理 L , 使 L 至少半满
- 4 如果 L 的最小键值发生改变, 则递归维护 L 的父节点的键值

例: 删除 $K = 13$ 的索引项 (无下溢)

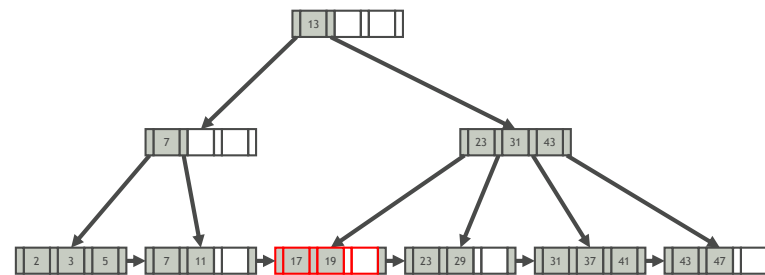


删除索引项

删除键为 K 的索引项

- 1 找到 K 所在的叶节点 L
- 2 从 L 中删除键值为 K 索引项
- 3 如果 L 发生下溢 (Underflow), 则需要处理 L , 使 L 至少半满
- 4 如果 L 的最小键值发生改变, 则递归维护 L 的父节点的键值

例: 删除 $K = 13$ 的索引项 (无下溢)

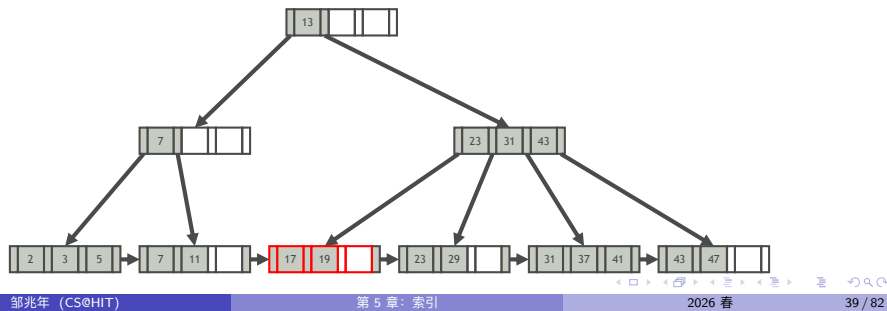


删除索引项

删除键为 K 的索引项

- 1 找到 K 所在的叶节点 L
- 2 从 L 中删除键值为 K 索引项
- 3 如果 L 发生下溢 (Underflow), 则需要处理 L , 使 L 至少半满
- 4 如果 L 的最小键值发生改变, 则递归维护 L 的父节点的键值

例: 删除 $K = 13$ 的索引项 (无下溢)

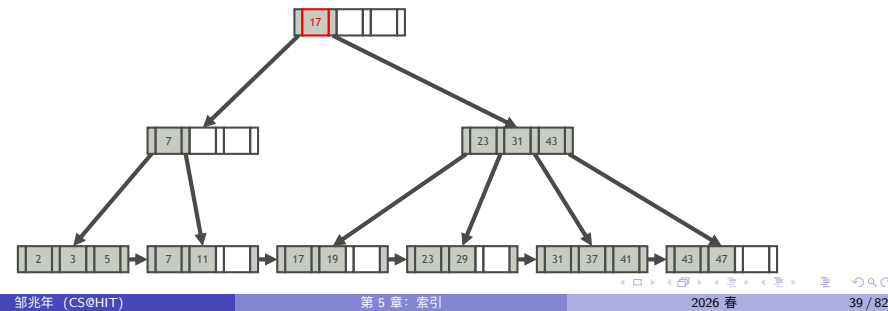


删除索引项

删除键为 K 的索引项

- 1 找到 K 所在的叶节点 L
- 2 从 L 中删除键值为 K 索引项
- 3 如果 L 发生下溢 (Underflow), 则需要处理 L , 使 L 至少半满
- 4 如果 L 的最小键值发生改变, 则递归维护 L 的父节点的键值

例: 删除 $K = 13$ 的索引项 (无下溢)

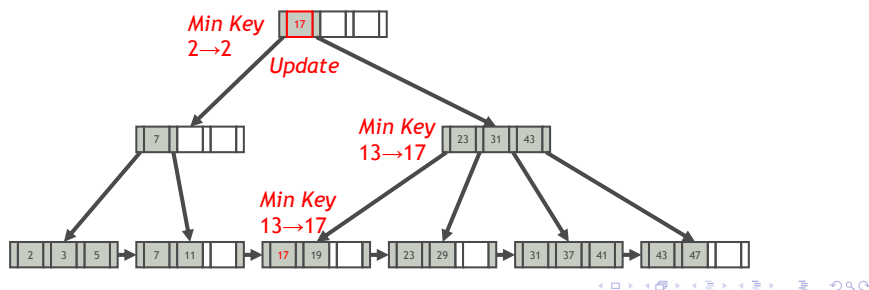


维护 B+ 树非叶节点键值的性质

定理 (B+ 树非叶节点键值的性质)

对于 $i \geq 1$, $Key[i-1] = Ptr[i]$ 指向的子树的最小键值

对于某节点 N , 如果以 N 为根的子树的最小键值发生变化, 则需要维护 N 的父亲节点中与指向 N 的指针 $Ptr[i]$ 关联的键值 $Key[i-1]$



节点下溢的处理方法

A 计划: “借”

- 节点再分配 (Redistribution): 从 L 的相邻兄弟节点借一个索引项, 使两者均至少半满
- 如果 L 向左右兄弟都能借, 则规定优先向左兄弟借

B 计划: “合”

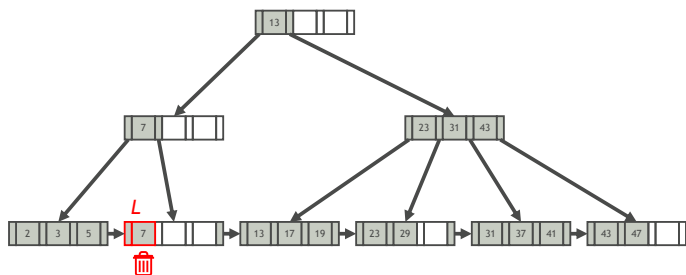
- 节点合并 (Colensing): 将 L 与其兄弟节点合并
- 如果 L 和左右兄弟都能合并, 则规定优先合并左兄弟

节点再分配 (Redistribution)

从叶节点 L 中删除索引项后发生下溢，并且能“借”

- 1 从 L 的相邻兄弟节点 L' 借一个索引项
- 2 如果 L 的最小键值发生变化，则更新 L 的祖先节点中的键值；如果 L' 的最小键值发生变化，则更新 L' 的祖先节点中的键值

例：删除 $K = 7$ 的索引项 (含节点重分布)

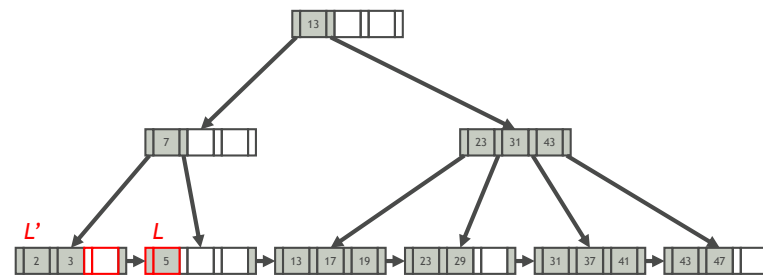


节点再分配 (Redistribution)

从叶节点 L 中删除索引项后发生下溢，并且能“借”

- 1 从 L 的相邻兄弟节点 L' 借一个索引项
- 2 如果 L 的最小键值发生变化，则更新 L 的祖先节点中的键值；如果 L' 的最小键值发生变化，则更新 L' 的祖先节点中的键值

例：删除 $K = 7$ 的索引项 (含节点重分布)

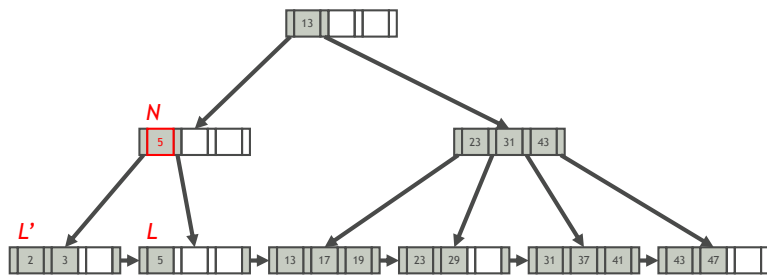


节点再分配 (Redistribution)

从叶节点 L 中删除索引项后发生下溢，并且能“借”

- 1 从 L 的相邻兄弟节点 L' 借一个索引项
- 2 如果 L 的最小键值发生变化，则更新 L 的祖先节点中的键值；如果 L' 的最小键值发生变化，则更新 L' 的祖先节点中的键值

例：删除 $K = 7$ 的索引项 (含节点重分布)

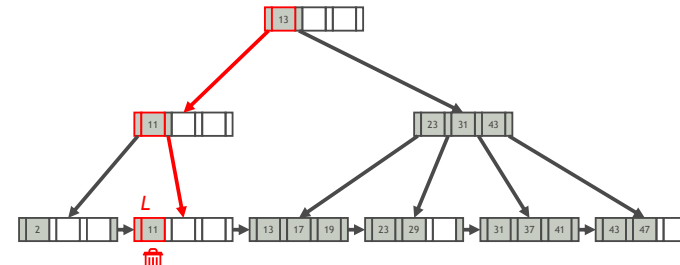


节点合并 (Colensing)

从叶节点 L 中删除索引项后发生下溢，却又不能“借”

- 1 将 L 与某相邻的兄弟节点 L' 合并，并从 L 的父节点 N 中删除相应的键值和指针。
- 2 如果 N 至少半满，则完成合并；否则， N 发生下溢，需处理 N ，使 N 至少半满
 - ▶ 如果 N 是根节点，且 N 中只有一个指针，则删除 N
 - ▶ 如果 N 是内节点，则对 N 递归地进行再分配或合并，使 N 至少半满

例：删除 $K = 11$ 的索引项 (含节点合并)

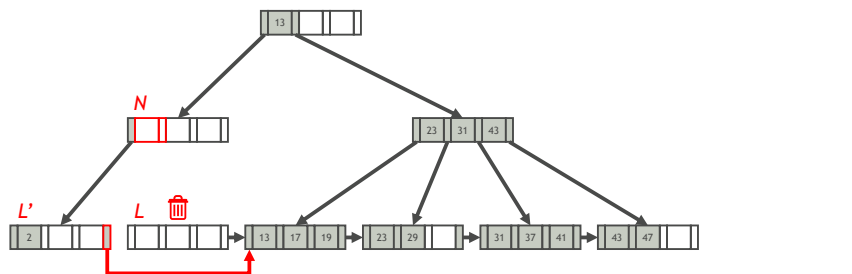


节点合并 (Colensing)

从叶节点 L 中删除索引项后发生下溢，却又不能“借”

- 1 将 L 与某相邻的兄弟节点 L' 合并，并从 L 的父节点 N 中删除相应的键值和指针。
- 2 如果 N 至少半满，则完成合并；否则， N 发生下溢，需处理 N ，使 N 至少半满
 - ▶ 如果 N 是根节点，且 N 中只有一个指针，则删除 N
 - ▶ 如果 N 是内节点，则对 N 递归地进行再分配或合并，使 N 至少半满

例：删除 $K = 11$ 的索引项（含节点合并）

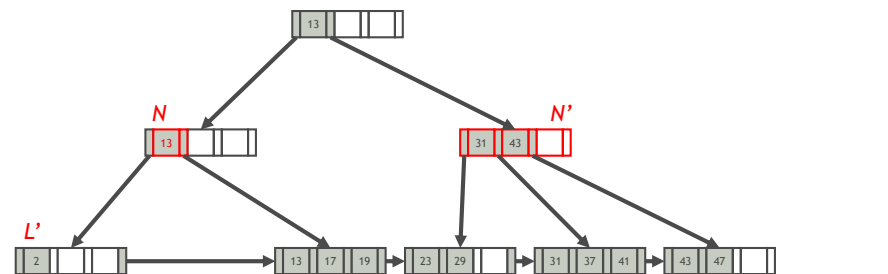


节点合并 (Colensing)

从叶节点 L 中删除索引项后发生下溢，却又不能“借”

- 1 将 L 与某相邻的兄弟节点 L' 合并，并从 L 的父节点 N 中删除相应的键值和指针。
- 2 如果 N 至少半满，则完成合并；否则， N 发生下溢，需处理 N ，使 N 至少半满
 - ▶ 如果 N 是根节点，且 N 中只有一个指针，则删除 N
 - ▶ 如果 N 是内节点，则对 N 递归地进行再分配或合并，使 N 至少半满

例：删除 $K = 11$ 的索引项（含节点合并）

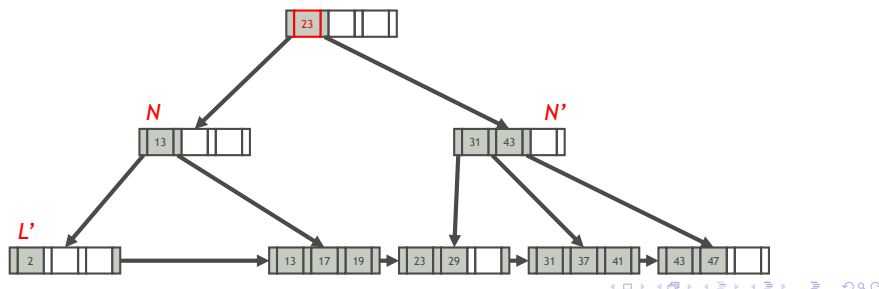


节点合并 (Colensing)

从叶节点 L 中删除索引项后发生下溢，却又不能“借”

- 1 将 L 与某相邻的兄弟节点 L' 合并，并从 L 的父节点 N 中删除相应的键值和指针。
- 2 如果 N 至少半满，则完成合并；否则， N 发生下溢，需处理 N ，使 N 至少半满
 - ▶ 如果 N 是根节点，且 N 中只有一个指针，则删除 N
 - ▶ 如果 N 是内节点，则对 N 递归地进行再分配或合并，使 N 至少半满

例：删除 $K = 11$ 的索引项（含节点合并）



实践学习：B+ 树可视化工具

使用 B+ 树可视化工具学习 B+ 树的知识

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>

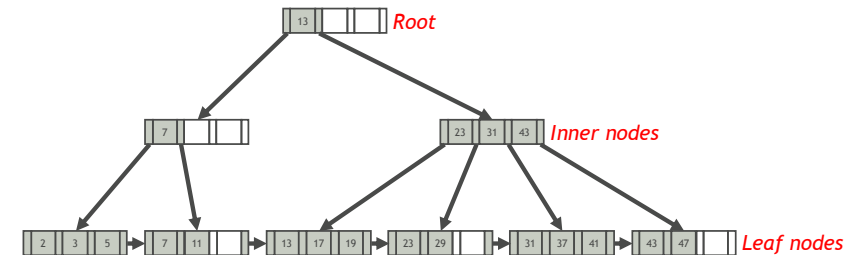
5.2.5 B+ 树的并发访问

B+ 树并发访问的问题

多线程并发读写 B+ 树可能产生错误

例 (并发读写相同叶节点可能产生错误)

- 线程 1: 查找 $K = 11$ 的索引项
- 线程 2: 插入 $K = 10$ 的索引项



B+ 树并发访问方法

- 粗粒度并发: 对根节点加锁
- 细粒度并发: 蟹行协议 (Crabbing Protocol)

B+ 树的粗粒度并发访问

每个线程执行下列步骤:

- 请求对根节点加锁 (执行读操作则加读锁, 执行写操作则加写锁)
- 如果加锁成功, 则继续访问 B+ 树; 否则, 等待, 直至获得根节点的锁
- 完成 B+ 树操作后, 释放根节点上的锁

B+ 树的细粒度并发访问（蟹行协议）

每个线程均遵守如下协议：

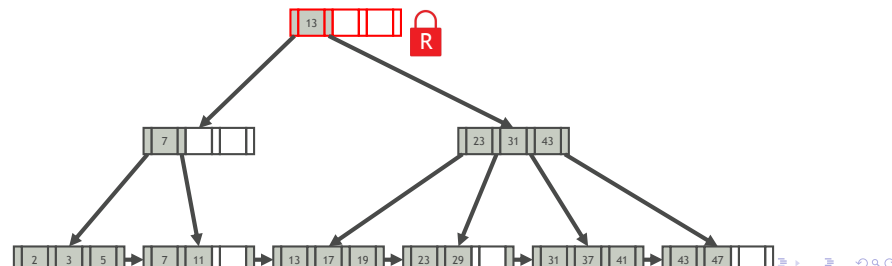
- 自上而下对节点请求加锁（蟹行）
- 加锁成功后，执行 B+ 树操作
- 自下而上对节点解锁（蟹行）

自上而下的请求加锁过程（查找）

每个线程在自上而下查找叶节点时，对可能发生冲突的节点请求加锁

- 1 请求对当前节点加锁（执行读操作则加读锁，执行写操作则加写锁）
- 2 如果加锁成功，则继续；否则，等待，直至获得当前节点的锁
- 3 如果当前节点安全（不会发生溢出或下溢），则释放其祖先节点上的锁（冲突不会发生在这些祖先节点上）
- 4 继续访问相应的儿子节点，执行步骤 1-3

例：查找 $K = 43$ 的索引项（加锁过程）

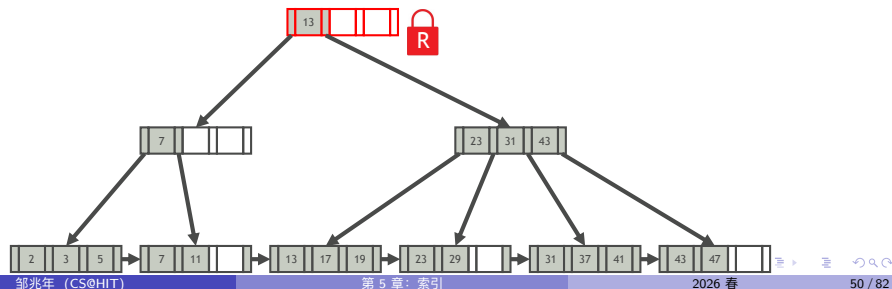


自上而下的请求加锁过程（查找）

每个线程在自上而下查找叶节点时，对可能发生冲突的节点请求加锁

- 1 请求对当前节点加锁（执行读操作则加读锁，执行写操作则加写锁）
- 2 如果加锁成功，则继续；否则，等待，直至获得当前节点的锁
- 3 如果当前节点安全（不会发生溢出或下溢），则释放其祖先节点上的锁（冲突不会发生在这些祖先节点上）
- 4 继续访问相应的儿子节点，执行步骤 1-3

例：查找 $K = 43$ 的索引项（加锁过程）

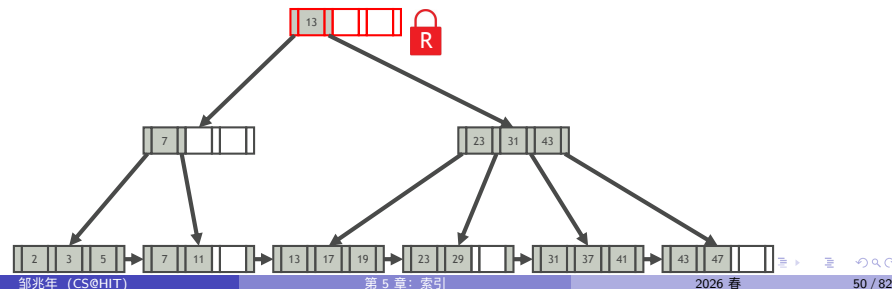


自上而下的请求加锁过程（查找）

每个线程在自上而下查找叶节点时，对可能发生冲突的节点请求加锁

- 1 请求对当前节点加锁（执行读操作则加读锁，执行写操作则加写锁）
- 2 如果加锁成功，则继续；否则，等待，直至获得当前节点的锁
- 3 如果当前节点安全（不会发生溢出或下溢），则释放其祖先节点上的锁（冲突不会发生在这些祖先节点上）
- 4 继续访问相应的儿子节点，执行步骤 1-3

例：查找 $K = 43$ 的索引项（加锁过程）

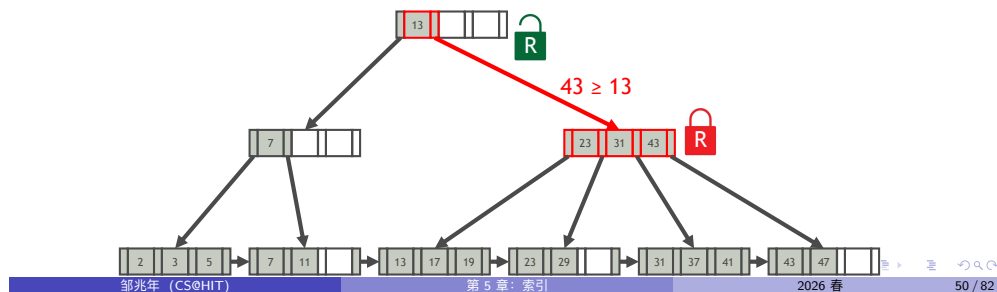


自上而下的请求加锁过程（查找）

每个线程在自上而下查找叶节点时，对可能发生冲突的节点请求加锁

- 1 请求对当前节点加锁（执行读操作则加读锁，执行写操作则加写锁）
- 2 如果加锁成功，则继续；否则，等待，直至获得当前节点的锁
- 3 如果当前节点安全（不会发生溢出或下溢），则释放其祖先节点上的锁（冲突不会发生在这些祖先节点上）
- 4 继续访问相应的儿子节点，执行步骤 1-3

例：查找 $K = 43$ 的索引项（加锁过程）

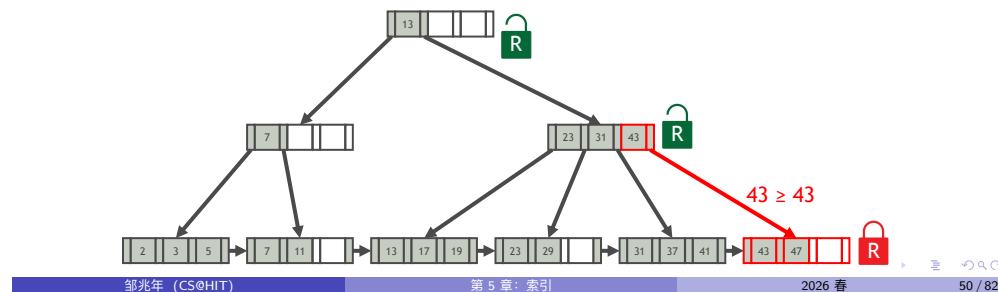


自上而下的请求加锁过程（查找）

每个线程在自上而下查找叶节点时，对可能发生冲突的节点请求加锁

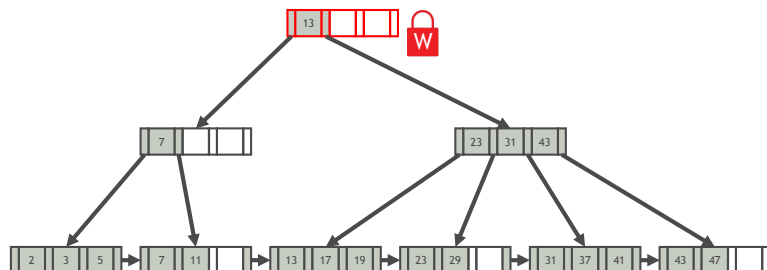
- 1 请求对当前节点加锁（执行读操作则加读锁，执行写操作则加写锁）
- 2 如果加锁成功，则继续；否则，等待，直至获得当前节点的锁
- 3 如果当前节点安全（不会发生溢出或下溢），则释放其祖先节点上的锁（冲突不会发生在这些祖先节点上）
- 4 继续访问相应的儿子节点，执行步骤 1-3

例：查找 $K = 43$ 的索引项（加锁过程）



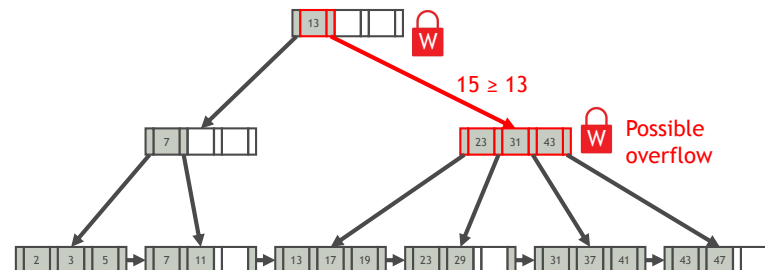
自上而下的请求加锁过程（插入）

例：插入 $K = 15$ 的索引项（加锁过程）



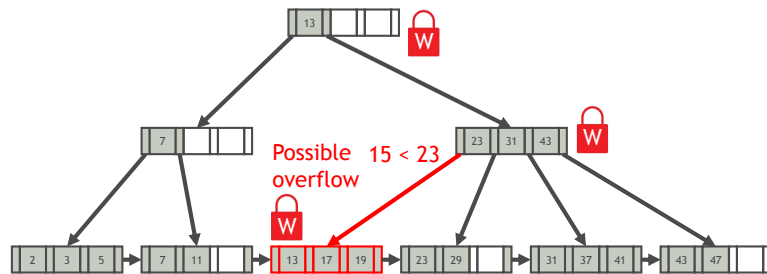
自上而下的请求加锁过程（插入）

例：插入 $K = 15$ 的索引项（加锁过程）



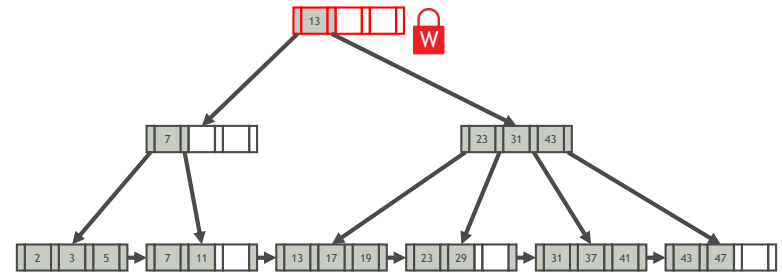
自上而下的请求加锁过程（插入）

例：插入 $K = 15$ 的索引项（加锁过程）



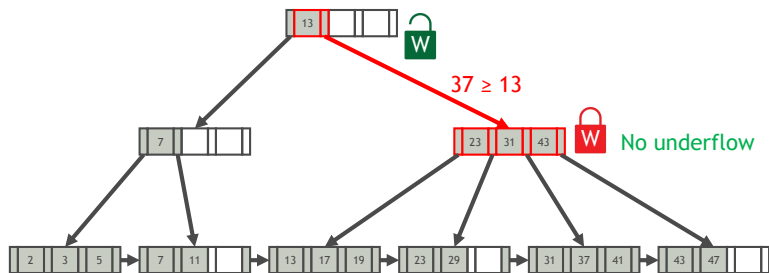
自上而下的请求加锁过程（删除）

例：删除 $K = 37$ 的索引项（加锁过程）



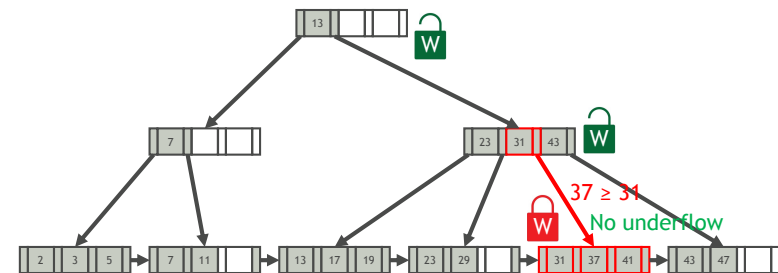
自上而下的请求加锁过程（删除）

例：删除 $K = 37$ 的索引项（加锁过程）



自上而下的请求加锁过程（删除）

例：删除 $K = 37$ 的索引项（加锁过程）

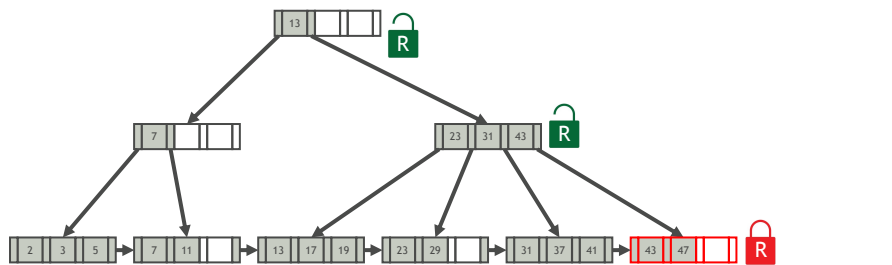


自底而上的解锁过程（查找）

在完成对叶节点的读写后，执行下列解锁步骤

- 1 释放当前节点上的锁
- 2 如果需要对当前节点的父节点进行分裂/再分配/合并，则在分裂/再分配/合并后，释放父节点上的锁；否则，释放当前节点祖先节点上的锁
- 3

例：查找 $K = 43$ 的索引项（解锁过程）

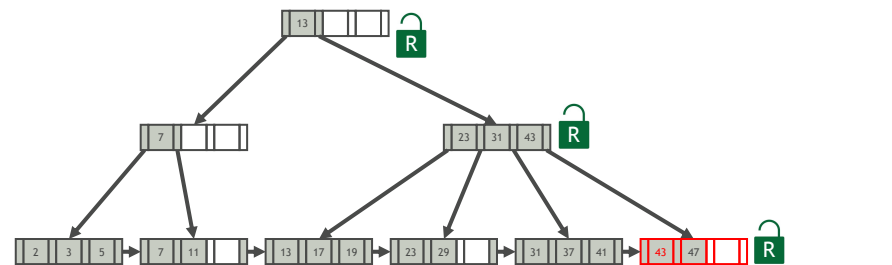


自底而上的解锁过程（查找）

在完成对叶节点的读写后，执行下列解锁步骤

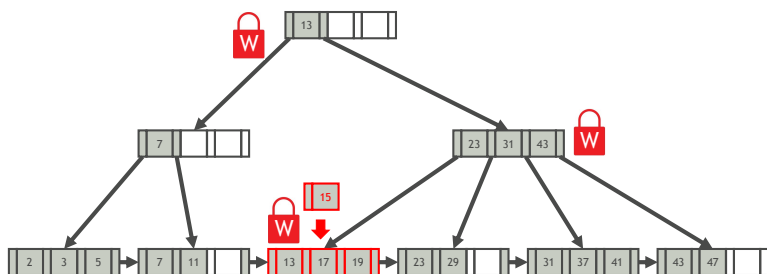
- 1 释放当前节点上的锁
- 2 如果需要对当前节点的父节点进行分裂/再分配/合并，则在分裂/再分配/合并后，释放父节点上的锁；否则，释放当前节点祖先节点上的锁
- 3

例：查找 $K = 43$ 的索引项（解锁过程）



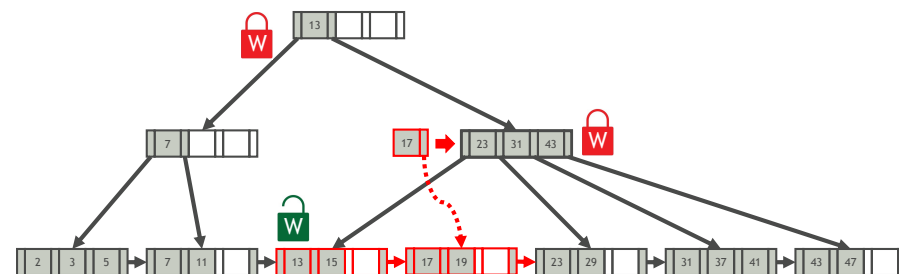
自底而上的解锁过程（插入）

例：插入 $K = 15$ 的索引项（解锁过程）



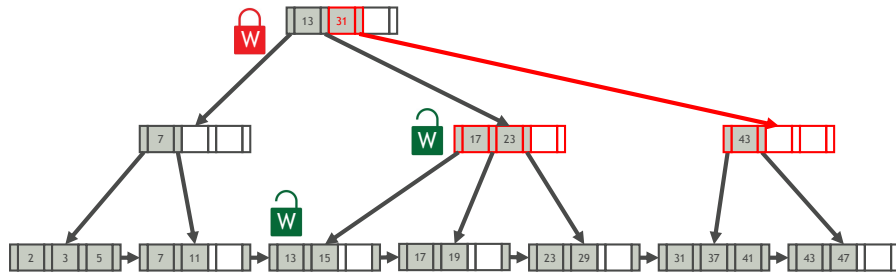
自底而上的解锁过程（插入）

例：插入 $K = 15$ 的索引项（解锁过程）



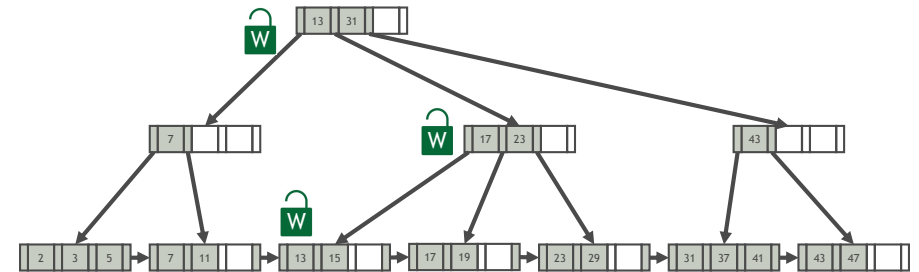
自底而上的解锁过程（插入）

例：插入 $K = 15$ 的索引项（解锁过程）



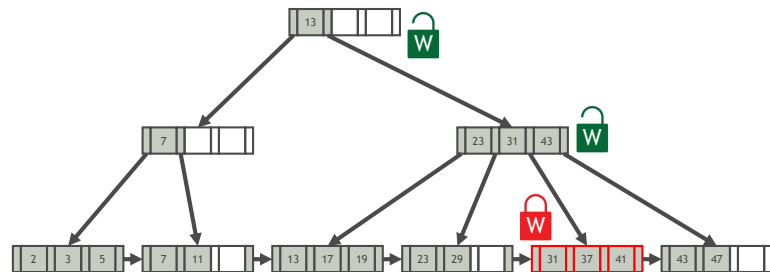
自底而上的解锁过程（插入）

例：插入 $K = 15$ 的索引项（解锁过程）



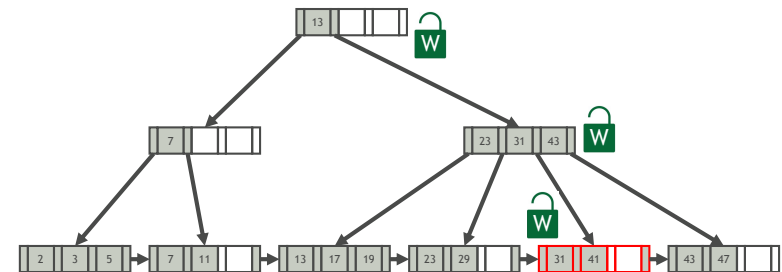
自底而上的解锁过程（删除）

例：删除 $K = 37$ 的索引项（解锁过程）



自底而上的解锁过程（删除）

例：删除 $K = 37$ 的索引项（解锁过程）



5.2.6 B+ 树的优化

B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点, 建立叶节点链表
- 3 创建一个空的内节点作为根, 并插入一个指针, 指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L , 向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针; 如果内节点溢出, 则分裂
- 5 重复第 4 步, 直至所有叶节点都插入 B+ 树为止

例: 在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



B+ 树的加载

自顶向下的方法

- 从一棵空的 B+ 树开始, 每次插入一个索引项
- 插入每个索引项都需要从根节点向下走到叶节点, 执行大量重复的计算

自底向上的方法

- 批量加载

B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点, 建立叶节点链表
- 3 创建一个空的内节点作为根, 并插入一个指针, 指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L , 向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针; 如果内节点溢出, 则分裂
- 5 重复第 4 步, 直至所有叶节点都插入 B+ 树为止

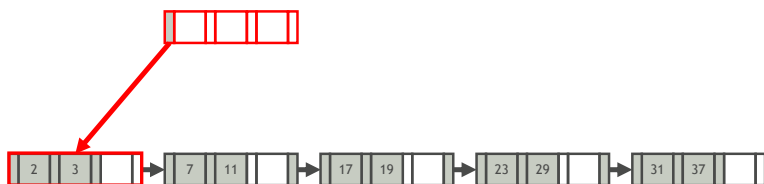
例: 在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点，建立叶节点链表
- 3 创建一个空的内节点作为根，并插入一个指针，指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L ，向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针；如果内节点溢出，则分裂
- 5 重复第 4 步，直至所有叶节点都插入 B+ 树为止

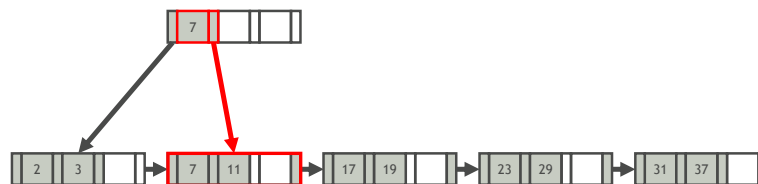
例：在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点，建立叶节点链表
- 3 创建一个空的内节点作为根，并插入一个指针，指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L ，向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针；如果内节点溢出，则分裂
- 5 重复第 4 步，直至所有叶节点都插入 B+ 树为止

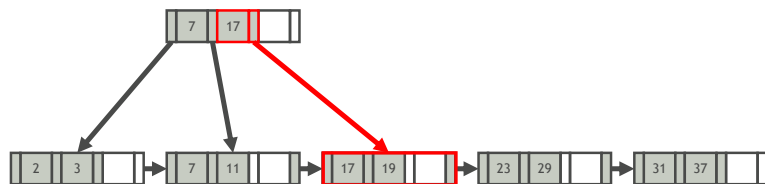
例：在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点，建立叶节点链表
- 3 创建一个空的内节点作为根，并插入一个指针，指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L ，向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针；如果内节点溢出，则分裂
- 5 重复第 4 步，直至所有叶节点都插入 B+ 树为止

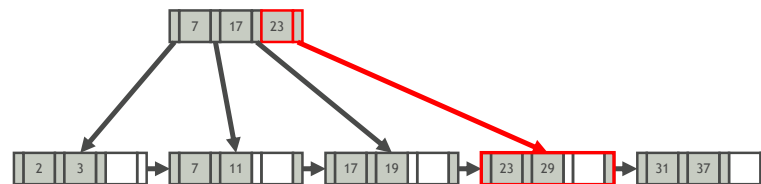
例：在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点，建立叶节点链表
- 3 创建一个空的内节点作为根，并插入一个指针，指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L ，向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针；如果内节点溢出，则分裂
- 5 重复第 4 步，直至所有叶节点都插入 B+ 树为止

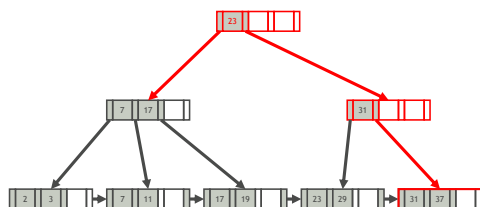
例：在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



B+ 树的批量加载 (Bulk Loading)

- 1 使用外存归并排序算法对所有页中的索引项排序
- 2 将每个页作为一个叶节点，建立叶节点链表
- 3 创建一个空的内节点作为根，并插入一个指针，指向第一个叶节点
- 4 对叶节点链表中的下一个叶节点 L ，向叶节点上层最右边的内节点插入 L 中最小的键及指向 L 的指针；如果内节点溢出，则分裂
- 5 重复第 4 步，直至所有叶节点都插入 B+ 树为止

例：在排好序的键 2, 3, 7, 11, 17, 19, 23, 29, 31, 37 上建立 B+ 树



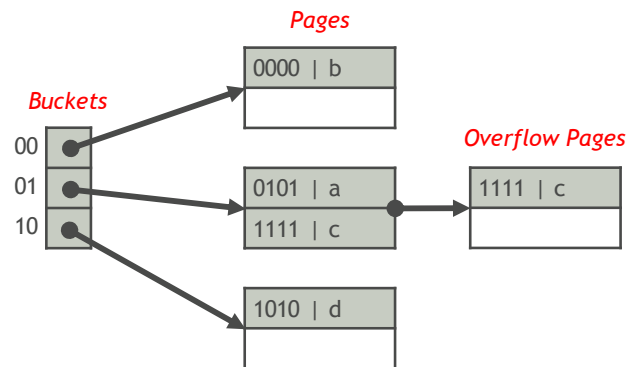
5.3 哈希索引

外存哈希表

一个外存哈希表包含多个桶 (Bucket)

- 设 $hash$ 是一个哈希函数，键值为 K 的索引项位于编号为 $hash(K)$ 的桶
- 每个桶中存放一个“指针” (页号)，指向存储该桶中索引项的页链表

5.3.1 外存哈希表



外存哈希表的分类

静态外存哈希表

- 桶的数量固定不变

动态外存哈希表

- 桶的数量动态变化，使每个桶中的索引项存储在大约 1 个页中
- 可扩展哈希表 (Extensible Hash Table)
- 线性哈希表 (Linear Hash Table)

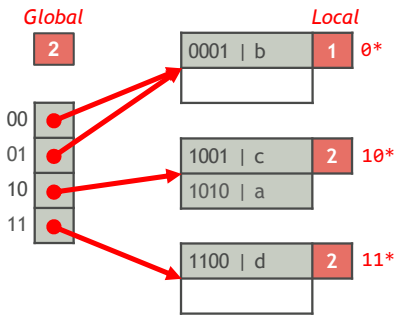
5.3.2 可扩展哈希表

可扩展哈希表 (Extensible Hash Table) 的结构

可扩展哈希表包含 2^i 个桶

- i : 全局深度 (Global Depth)
- 键值为 K 的索引项位于编号等于 $hash(K)$ 的前 i 位的桶

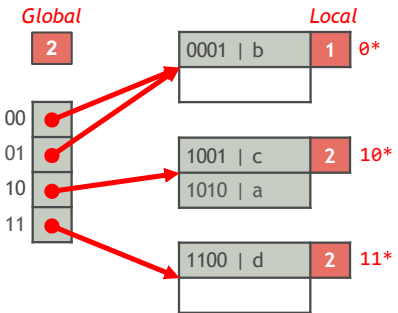
例: $hash(a) = 1010, hash(b) = 0001, hash(c) = 1001, hash(d) = 1100$



可扩展哈希表的结构

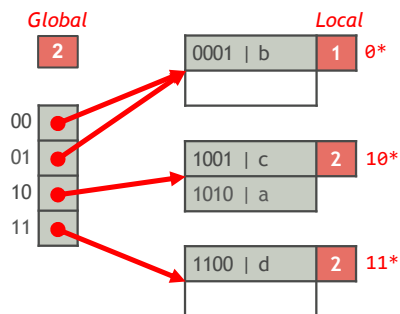
每个桶中存放一个“指针” (页号)，指向存储该桶中索引项的页

- 桶没有溢出页 (Overflow Page)
- 如果容纳得下，多个相邻桶中的全部索引项可以存入同一个页
- 每个页记录一个局部深度 (Local Depth) j ，该页中的全部索引项键值 K 的 $hash(K)$ 的前 j 位相同，用于标识这些索引项都存于这个页



可扩展哈希表的性质

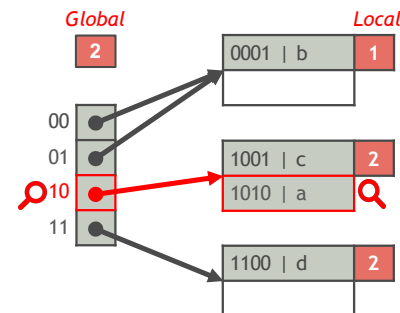
- 桶数 = $2^{\text{global_depth}}$
- 全局深度 $\geq$ 页的局部深度
- 多个桶共享一个页当且仅当这个页的局部深度小于全局深度
- 设页的局部深度为 j ，则页中存放的全部索引项键值 K 的 $\text{hash}(K)$ 前 j 位相同



查找索引项

- 确定索引项所属的桶
- 在桶指向的页中查找索引项

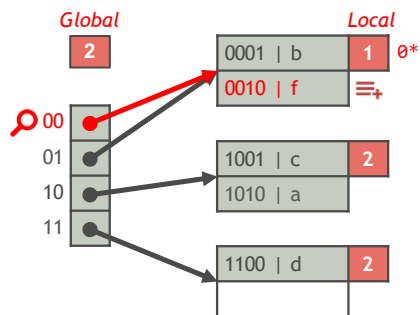
例: 查找 $K = a$, $\text{hash}(a) = 1010$



插入索引项

- 找到索引项应被插入的页 P
- 如果 P 中有足够的空闲空间，则将索引项插入 P 中；否则， P 溢出，分裂 P

例: 插入 $K = f$, $\text{hash}(f) = 0010$ (无溢出)

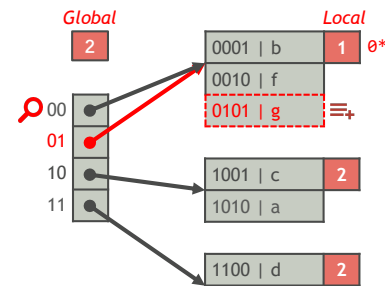


页溢出的处理 (情况 1)

情况 1: P 溢出且 P 的局部深度小于全局深度

- 将 P 的局部深度 j 加 1
- 创建一个新页 P' ，其局部深度等于 P 的局部深度
- 根据键的哈希值的前 j 位，将 P 中索引项和被插入的索引项在 P 和 P' 中重新分配
- 更新指向 P 的桶中的指针

例: 插入 $K = g$, $\text{hash}(g) = 0101$ (含溢出情况 1)

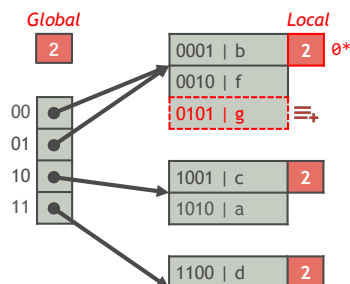


页溢出的处理 (情况 1)

情况 1: P 溢出且 P 的局部深度小于全局深度

- 1 将 P 的局部深度 j 加 1
- 2 创建一个新页 P' , 其局部深度等于 P 的局部深度
- 3 根据键的哈希值的前 j 位, 将 P 中索引项和被插入的索引项在 P 和 P' 中重新分配
- 4 更新指向 P 的桶中的指针

例: 插入 $K = g$, $\text{hash}(g) = 0101$ (含溢出情况 1)

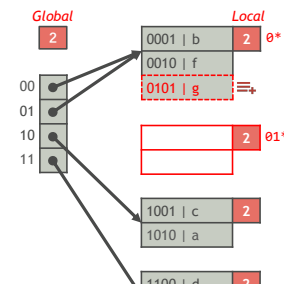


页溢出的处理 (情况 1)

情况 1: P 溢出且 P 的局部深度小于全局深度

- 1 将 P 的局部深度 j 加 1
- 2 创建一个新页 P' , 其局部深度等于 P 的局部深度
- 3 根据键的哈希值的前 j 位, 将 P 中索引项和被插入的索引项在 P 和 P' 中重新分配
- 4 更新指向 P 的桶中的指针

例: 插入 $K = g$, $\text{hash}(g) = 0101$ (含溢出情况 1)

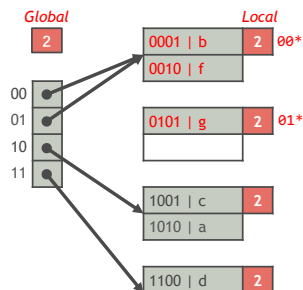


页溢出的处理 (情况 1)

情况 1: P 溢出且 P 的局部深度小于全局深度

- 1 将 P 的局部深度 j 加 1
- 2 创建一个新页 P' , 其局部深度等于 P 的局部深度
- 3 根据键的哈希值的前 j 位, 将 P 中索引项和被插入的索引项在 P 和 P' 中重新分配
- 4 更新指向 P 的桶中的指针

例: 插入 $K = g$, $\text{hash}(g) = 0101$ (含溢出情况 1)

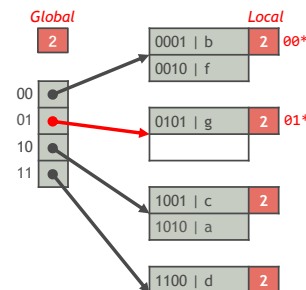


页溢出的处理 (情况 1)

情况 1: P 溢出且 P 的局部深度小于全局深度

- 1 将 P 的局部深度 j 加 1
- 2 创建一个新页 P' , 其局部深度等于 P 的局部深度
- 3 根据键的哈希值的前 j 位, 将 P 中索引项和被插入的索引项在 P 和 P' 中重新分配
- 4 更新指向 P 的桶中的指针

例: 插入 $K = g$, $\text{hash}(g) = 0101$ (含溢出情况 1)

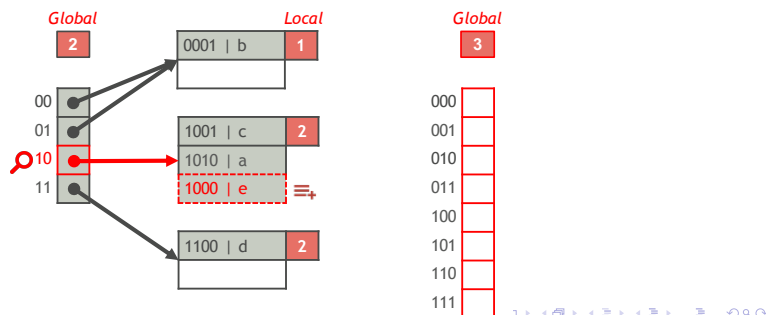


页溢出的处理 (情况 2)

情况 2: P 溢出且 P 的局部深度等于全局深度 (含溢出情况 2)

- 1 将全局深度加 1，即桶的数量翻倍
- 2 更新每个桶中的页指针
- 3 对于 P_i ，使用前面介绍的方法分裂 P_i

例：插入 $K = e$, $hash(e) = 1000$ (含溢出情况 2)

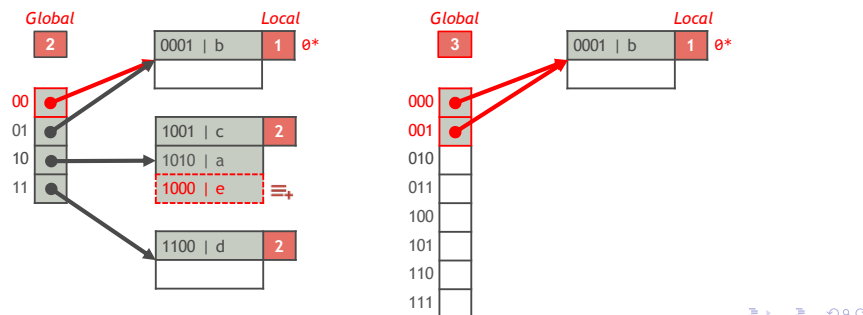


页溢出的处理 (情况 2)

情况 2: P 溢出且 P 的局部深度等于全局深度 (含溢出情况 2)

- ① 将全局深度加 1，即桶的数量翻倍
- ② 更新每个桶中的页指针
- ③ 对于 P_i ，使用前面介绍的方法分裂 P_i

例：插入 $K = e$, $hash(e) = 1000$ (含溢出情况 2)

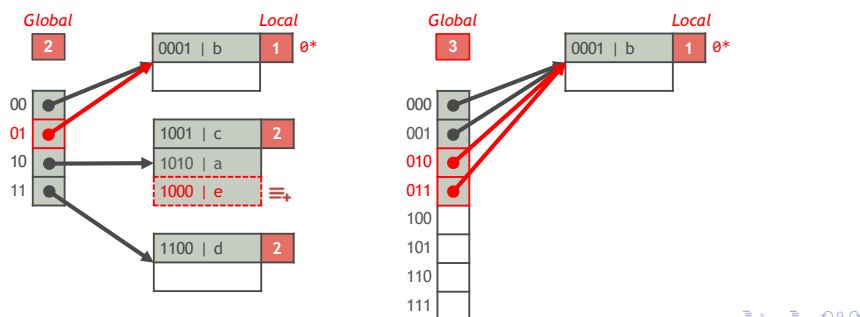


页溢出的处理 (情况 2)

情况 2: P 溢出且 P 的局部深度等于全局深度 (含溢出情况 2)

- 1 将全局深度加 1，即桶的数量翻倍
- 2 更新每个桶中的页指针
- 3 对于 P ，使用前面介绍的方法分裂 P

例：插入 $K = e$, $hash(e) = 1000$ (含溢出情况 2)

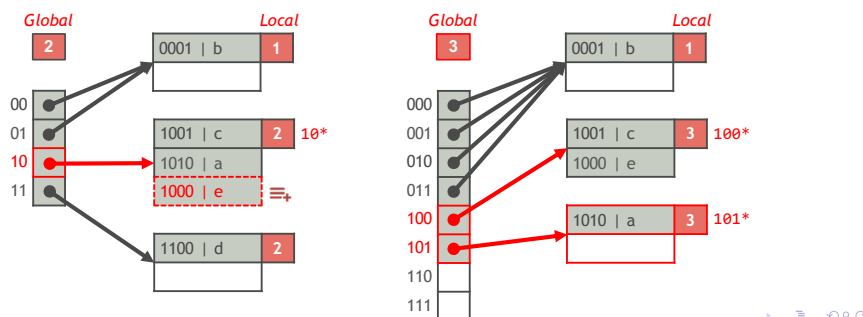


页溢出的处理 (情况 2)

情况 2: P 溢出且 P 的局部深度等于全局深度 (含溢出情况 2)

- 1 将全局深度加 1，即桶的数量翻倍
- 2 更新每个桶中的页指针
- 3 对于 P ，使用前面介绍的方法分裂 P

例：插入 $K = e$, $hash(e) = 1000$ (含溢出情况 2)

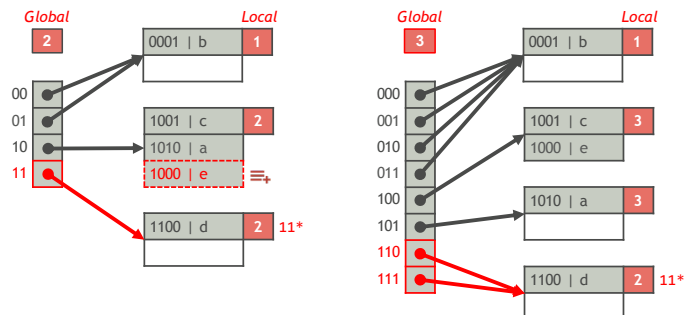


页溢出的处理 (情况 2)

情况 2: P 溢出且 P 的局部深度等于全局深度 (含溢出情况 2)

- 1 将全局深度加 1, 即桶的数量翻倍
- 2 更新每个桶中的页指针
- 3 对于 P , 使用前面介绍的方法分裂 P

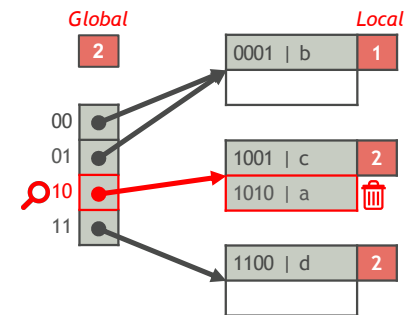
例: 插入 $K = e$, $\text{hash}(e) = 1000$ (含溢出情况 2)



删除索引项

- 1 找到索引项所在的页
- 2 从页中删除索引项

例: 删除 $K = a$, $\text{hash}(a) = 1010$



删除索引项后, 是否需要合并页?

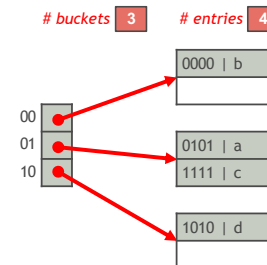
5.3.3 线性哈希表

线性哈希表 (Linear Hash Tables)

线性哈希表包含 n 个桶

- 每个桶中保存一个“指针” (页号), 指向存储该桶中索引项的页链表
- 假设每个页最多存储 b 个索引项, 每个桶只有 1 个页, 则线性哈希表中最多存储 θbn 个索引项, 其中 $0 < \theta < 1$ 是一个阈值
- 记录线性哈希表中桶的数量 ($\# \text{buckets}$) 和索引项的数量 ($\# \text{entries}$)

例: $b = 2$, $\theta = 0.85$



线性哈希表的哈希方案 (Hashing Scheme)

- 设桶号为 $0, 1, \dots, n-1$
- 令 $m = 2^{\lceil \log_2 n \rceil}$, 因此 $m \leq n < 2m$
- 对于键值为 K 的索引项, 如果 $\text{hash}(K) \bmod 2m < n$, 则该索引项位于编号为 $\text{hash}(K) \bmod 2m$ 的桶; 否则, 该索引项位于编号为 $\text{hash}(K) \bmod m$ 的桶

例 (线性哈希表的哈希方案)

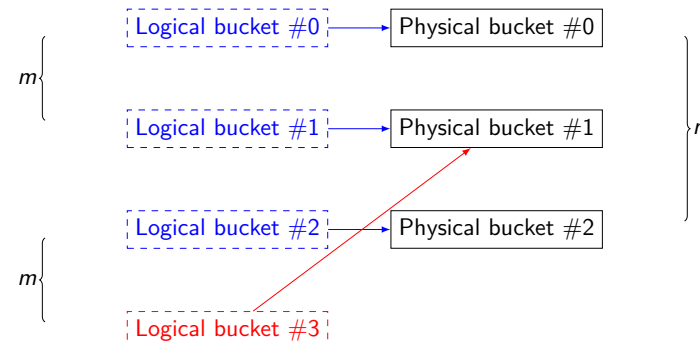
设 $n = 3$, 有 $m = 2$

Bucket #0	$\text{hash}(K) = 0, 4, 8, \dots$
Bucket #1	$\text{hash}(K) = 1, \textcolor{red}{3}, 5, \textcolor{red}{7}, 9, \dots$
Bucket #2	$\text{hash}(K) = 2, 6, 10, \dots$

桶的负载不均衡

线性哈希表哈希方案的解释

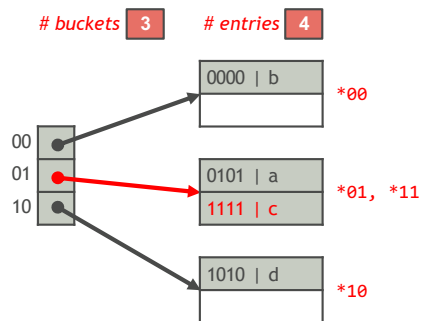
- 逻辑上有 $2m$ 个桶, 物理上有 n 个桶, $n < 2m$
- 键值为 K 的索引项的逻辑桶号 $b(K) = \text{hash}(K) \bmod 2m$
- 如果 $b(K) < n$, 则该索引项属于 $b(K)$ 号物理桶
- 如果 $b(K) \geq n$, $b(K)$ 号物理桶不存在, 则该索引项被放入 $b(K) \bmod m$ 号物理桶



查找索引项

- 确定索引项所属的桶
- 在桶指向的页链表中查找索引项

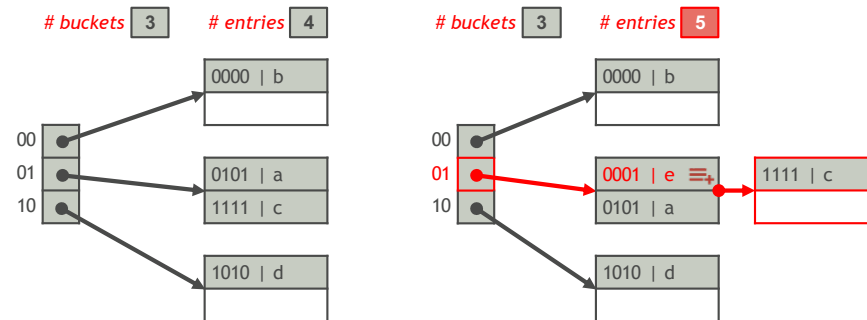
例: 查找 $K = c$, $\text{hash}(c) = 1111$



插入索引项

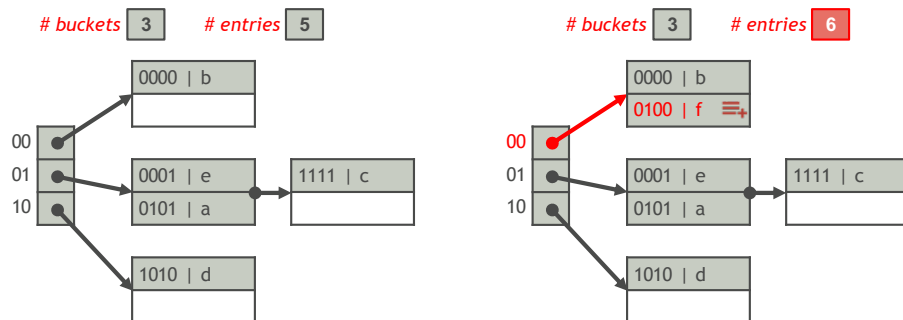
- 将索引项插入它所属的桶 B
- 将索引项的数量 ($\#entries$) 加 1
- 如果 $\#entries \leq \theta bn$, 则插入完成; 否则, 将桶的数量 ($\#buckets$) 加 1, 按照哈希方案重新分配 B 中的索引项

例: 插入 $\text{hash}(e) = 0001$, $\theta = 0.85$



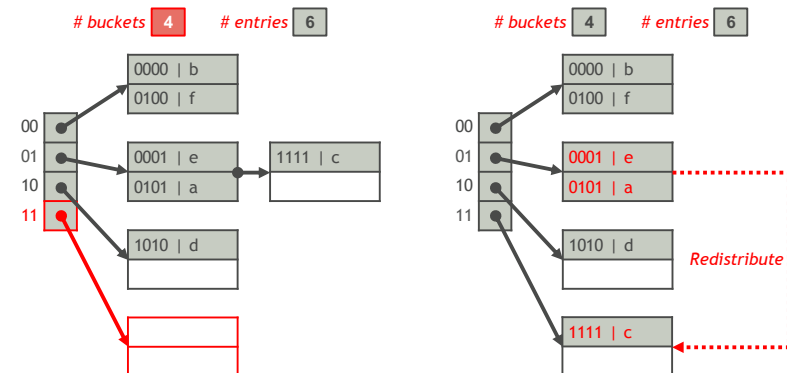
插入索引项（含重分配）

例：插入 $hash(f) = 0100, \theta = 0.85$ （含重分配）



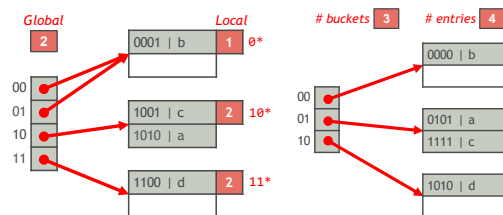
插入索引项（含重分配）

例：插入 $hash(f) = 0100, \theta = 0.85$ （含重分配）



可扩展哈希表 VS 线性哈希表

	可扩展哈希表	线性哈希表
桶的数量	2^{global_depth}	n
是否有溢出页	无	有
哈希方案	$hash(K)$ 的前 $global_depth$ 位	$hash(K) \bmod 2m$ 或 $hash(K) \bmod m$
页分裂条件	页发生溢出	$\#entries > \theta bn$
增加桶的方法	桶数翻倍 ($global_depth$ 加 1)	桶数加 1



探究式学习：基于人工智能的数据库索引

使用深度学习技术，优化索引结构的设计

- X. Zhou, C. Chai, G. Li, J. Sun. **Database Meets Artificial Intelligence: A Survey.** *IEEE Transactions on Knowledge and Data Engineering*, 34(3):1096–1116, 2022. (Section 2.3.1 “Learned Data Structure”)

总结

① 5.1 索引的概念

② 5.2 有序索引

- 5.2.1 B+ 树的结构
- 5.2.2 B+ 树查找
- 5.2.3 B+ 树插入
- 5.2.4 B+ 树删除
- 5.2.5 B+ 树的并发访问
- 5.2.6 B+ 树的优化

3 5.3 哈希索引

- 5.3.1 外存哈希表
- 5.3.2 可扩展哈希表
- 5.3.3 线性哈希表

致谢

2026 版以前

- 感谢詹儒彦 (1190202307)、金彦铮 (1190200418)、杨宇辰 (1190300611)、蔡思娣 (1190201925) 同学指出课件中的错误